

External agents on Front

A scoped, auditable way for third-party AI to read and act on Front

Developer Platform · PRD walkthrough

THE PROBLEM

Agents borrow a teammate's identity, not their own

What exists

OAuth-based access with real read, write, and send scopes at the tool level.

What's missing

An identity for the agent itself, distinct from the teammate it's impersonating.

Every connection today authenticates as a specific teammate and inherits that person's exact permissions. There's no admin-managed way to grant an agent its own narrower, revocable access, and any activity record shows the teammate, not whether the agent or the person actually acted.

WHO THIS IS FOR

Three jobs, one platform decision

Customer (Instructure)

Grant only the access an agent needs. See what it did. Revoke it without engineering help.

Partner (Aircall)

One scope model a customer's admin grants directly, not one tied to whichever teammate authorized it.

Third-party client

(Claude Desktop)

The same access pattern, working identically at every customer that installs it.

Real, public examples: a Front customer story, a named Front Integration Partner, and a generic MCP client that needs no partnership to connect at all.

HIGHEST-LEVERAGE USE CASES

Read, draft, triage, trigger

USE CASE	RISK	WHY IT MATTERS
Read context	Low	The on-ramp for everything else the agent does
Draft a reply	Low, human sends	Highest leverage per unit of risk
Triage / classify	Medium, recoverable	Value compounds at volume
Trigger a defined workflow	Medium, scoped	Reuses automation Front already trusts

OUT OF SCOPE, AND NON-GOALS

What's deferred, and what's permanent

Not now (deferred)

Autonomous send

Partner marketplace or directory

SDKs beyond raw MCP tool calls

Not ever (non-goals)

Self-expanding scopes

Cross-workspace or cross-tenant access

A non-MCP transport

Each deferred item has a real revisit trigger tied to beta data, not a fixed date. Non-goals would undermine the safety model itself, not just add scope to it.

What ships first

Agent connection as a named, scoped principal, not a generic token

MCP tools gated by connection scope, not an impersonated role

Human-in-the-loop send, a structural gate, not a UI convention

Audit log of every scoped action: connection, scope, target, time

One-click revocation, effective immediately

No autonomous send, no self-expanding scopes, no marketplace, in v1

ROADMAP

Draft-and-approve first, autonomy earned later

PHASE	DURATION	SCOPE
0	4-6 wks	Scope taxonomy and data model with security and infra
1	8-10 wks	Connect/consent UI, audit log, read + draft tools. Closed beta.
2	6-8 wks	Tag/assign and workflow-trigger tools. Open beta, self-serve.
3	GA	Decide v1.1 auto-send opt-in on measured trust, not a date

RISKS

What could go wrong, and the mitigation

RISK	MITIGATION
A scoped agent is still a meaningful blast radius	Team/inbox-level default scopes, per-connection rate limits
Reps ignore drafts, adoption stalls	Measure edit-distance and send-rate in beta; make accept faster than writing
MCP auth patterns still stabilizing industry-wide	Version the server interface from day one, track the spec's auth working group
DPA's may not contemplate a third-party agent	Legal and compliance in Phase 0; developer data-handling attestation

Four, matching the brief's own categories

Adoption

30% of API-plan workspaces create a connection within 90 days of GA.

Time to first integration

Median under 30 minutes, credentials to first sandbox tool call.

Developer satisfaction

8/10+ on scope-model clarity in the post-beta survey.

Reliability

99.9% MCP server availability, P95 tool-call latency under 500ms.

MCP as the door, not a new house

External agent

→ MCP tool call →

Front MCP server

scope check

→

Front's internal services

same path the UI uses

Writes go through the same command/event pipeline Front already uses for its own automation. Nothing new at the domain level, only a new, scoped entry point into it.

[SendReply](#) is the one command no agent credential can ever reach, see the event model.

TECHNICAL TRADE-OFFS

What I'd work through with engineering

Coupling vs. gateway: fast to ship vs. safe to evolve independently

Sync vs. async tool calls: simple today vs. what long-running actions need

Fine-grained vs. bundled scopes: least privilege vs. easy to reason about

Per-connection vs. per-workspace rate limits: predictable vs. infra-safe

THE HARDEST ASSUMPTION

Human review is enough to de-risk launch

If customers push for auto-send in week one because draft-and-approve doesn't save enough time, the fix is that the permission and audit layer, the actual safety mechanism, was already built robust enough to support it. Extending to v1.1 becomes a scope change, not a re-architecture.